Zenvyrolabs Voice Studio Internship
Task Report

Tehami Azmat

2 August 2026

Contents

1 Introduction

This report documents the engineering work completed for the Zenvyrolabs Advanced Voice Studio Internship assignment. The project involved taking an early-stage voice-cloning prototype – built on F5-TTS (voice cloning), Edge-TTS (native-language text-to-speech), and RVC (voice conversion) – and engineering it into a containerized, production-ready application. The assignment was structured into four tasks: Dockerization, podcast script parsing with Hindi/Urdu pronunciation routing, an audio preprocessing pipeline for ML training data, and a final demonstration deliverable.

2 Task 1: Dockerization

2.1 Objective

Package the application into a Docker container with a complete, working `requirements.txt`, replacing the fragile local setup relied upon in the original prototype.

2.2 Work Completed

A `Dockerfile` and `docker-compose.yml` were written from scratch. Dependencies were split into two files – `requirements-core.txt` and `requirements-rvc.txt` – installed in separate stages to resolve an unsolvable version conflict (Section 4.1). Persistent data (saved voices, RVC models, training data) was mounted as Docker volumes so it survives container restarts.

2.3 Outcome

The container builds successfully and the Gradio web interface launches cleanly on port 7860, accessible via the host browser.

3 Task 2: Podcast Script Parsing and Hindi/Urdu Routing

3.1 Objective

Harden the multi-voice podcast script parser against malformed input, and route Hindi/Urdu dialogue lines through native-pronunciation text-to-speech (Edge-TTS) instead of the English-oriented voice-cloning engine (F5-TTS), while smoothing transitions between spoken lines.

3.2 Work Completed

- Rewrote `parse_podcast_script()` to return descriptive warnings for malformed lines instead of silently skipping or crashing.
- Added Devanagari/Arabic script detection (`contains_hindi_urdu_script()`) to route individual dialogue lines to the appropriate engine.
- Replaced the fixed silence gap between lines with an audio crossfade.
- Fixed a voice-mapping validation bug that incorrectly required a saved voice clone for characters whose lines were entirely Hindi/Urdu.

- Diagnosed and fixed a reference-audio/reference-text mismatch causing garbled F5-TTS output on long source recordings (Section 4.2).
- Corrected a low `nfe_step` value that was causing unnaturally rushed speech pacing.

3.3 Outcome

The podcast pipeline was verified end-to-end with a mixed English/Hindi test script, correctly routing lines through the appropriate engine and producing a coherent, correctly-paced final audio file.

4 Task 3: Audio Preprocessing Pipeline

4.1 Objective

Add noise reduction and silence trimming to the existing volume-normalization and chunking pipeline used to prepare training data.

4.2 Work Completed

Noise reduction (spectral gating, via the `noisereduce` library) and silence trimming (via `pydub.silence.detect_n` were implemented and integrated into `preprocess_training_audio()`, ordered so that noise reduction runs before silence detection and volume normalization for more reliable results.

4.3 Outcome

Not fully functional at time of submission. Testing surfaced a deep, upstream incompatibility between the `numba` JIT compiler and `librosa`'s internal resampling function (Section 4.3), which crashes the feature regardless of application-level code correctness. This is documented as a known limitation.

5 Task 4: Demonstration

5.1 Objective

Produce a demonstration podcast using the completed pipeline and record a walkthrough video.

5.2 Work Completed

A demo script was written featuring both saved voices in a mixed English/Hindi conversation, specifically designed to exercise the Task 2 routing logic on camera. The podcast was generated successfully through the working pipeline.

6 Major Technical Challenges and Solutions

6.1 The `hydra-core` Dependency Conflict

`f5-tts` requires `hydra-core>=1.3.0`; `fairseq` (a transitive dependency of `rvc-python`) requires `hydra-core<1.1`. These are genuinely incompatible constraints that no version pin can satisfy

simultaneously in a single `pip` resolution pass. **Solution:** dependencies were split into two requirements files. The RVC/fairseq chain is installed first with `pip install --no-deps`, which installs the exact packages listed without letting `pip` inspect or resolve their declared sub-dependencies – meaning fairseq's outdated `hydra-core` pin is never seen by the resolver. The core dependencies (including `f5-tts`) are installed afterward normally, free to pull in the modern `hydra-core` they require.

6.2 Reference Audio/Text Mismatch (F5-TTS)

One saved reference voice consisted of an 18-minute recording, while F5-TTS's inference code always trims reference audio to the first 8 seconds. The saved reference transcript, however, described the full recording rather than just those first 8 seconds, causing F5-TTS to receive text that did not match the audio it was actually conditioned on – producing garbled, artifact-laden output. **Solution:** the code was changed to always re-transcribe the actual trimmed 8-second clip via Whisper immediately before generation whenever the source audio exceeds 8 seconds, guaranteeing the reference text and audio always correspond.

6.3 The `numba`/`librosa` Incompatibility

Attempting to use `noisereduce` (which depends on `librosa`) inside the Docker container consistently failed with `cannot import name 'stft' from 'librosa'`. Diagnosis required extensive isolated testing: standalone scripts calling the same functions succeeded reliably, which excluded initial theories including multiprocessing behavior and thread-safety races in `librosa`'s lazy module loading. Forcing an eager import of `librosa` at application startup surfaced the true underlying error: `numba`, the JIT compiler `librosa` uses internally to accelerate a resampling routine, fails to compile that routine correctly under the installed `numba` version, producing an internal `AttributeError` deep inside `numba`'s own type-inference engine. An attempted version pin (`numba==0.58.1`) did not resolve the issue. This remains an open, upstream-level compatibility problem between specific `numba` and `librosa` releases, unrelated to application code, and is documented as a known limitation for this submission.

7 Conclusion

Tasks 1, 2, and 4 were completed and verified working. Task 3 was implemented in full but is blocked by an unresolved third-party library incompatibility, documented above with a full diagnostic trail. The project involved substantial real-world dependency-resolution engineering across a stack combining PyTorch, F5-TTS, RVC/fairseq, and Whisper – surfacing and resolving several genuine, non-trivial version conflicts along the way.